

RESEARCH PROPOSAL · HANDS-ON QUANTUM COMPUTING PROJECT

Quantum-Assisted Emergency Vehicle Routing for Congested Urban Roads

*A QUBO-QAOA framework benchmarked against classical optimization,
with a case study on Dhaka, Bangladesh*

Prepared by: [Your Name]

Affiliation: [Department / Institution]

Field: Quantum Optimization · Operations Research

Date: 26 August 2026

Version: 1.0 (Draft)

Contents

Abstract	3
1 Introduction & Motivation	3
1.1 The Dhaka congestion problem.....	3
1.2 Why routing here is genuinely hard.....	3
1.3 Why quantum, and why now.....	4
1.4 Learning framing.....	4
2 Problem Statement	4
3 Research Objectives & Questions	4
3.1 Objectives.....	4
3.2 Research questions.....	5
4 Background & Related Work	5
4.1 QUBO and the Ising model.....	5
4.2 The Quantum Approximate Optimization Algorithm.....	5
4.3 Quantum routing and classical optimisation.....	6
5 Methodology	6
5.1 Data acquisition & instance generation.....	6
5.2 Mathematical formulation.....	6
5.3 Classical baselines.....	7
5.4 Quantum approach (QAOA).....	7
5.5 Noise-aware transpilation & error mitigation.....	8
5.6 Hardware execution plan (IBM Quantum).....	8
6 Evaluation Plan & Metrics	8
7 Hands-On Learning Roadmap & Timeline	9
8 Expected Outcomes & Contributions	9
9 Risks, Limitations & Mitigations	10
10 Tools, Software & Resources	10
11 References (selected)	11
Appendix A • Illustrative QAOA skeleton (Qiskit)	12

Abstract

Emergency response times in dense, congested megacities such as Dhaka are frequently degraded by unpredictable traffic, incomplete road information, and the combinatorial difficulty of choosing optimal routes and vehicle–incident assignments in real time. This proposal sets out a combined research and learning program to investigate whether near-term quantum optimization — specifically the Quantum Approximate Optimization Algorithm (QAOA) applied to Quadratic Unconstrained Binary Optimization (QUBO) formulations — can produce competitive routing solutions for emergency vehicles on realistic Bangladeshi road networks.

The work will (i) construct a congestion-aware road-network dataset for a district of Dhaka from OpenStreetMap; (ii) formulate the routing and vehicle-assignment problems as QUBOs; (iii) establish strong classical baselines using Dijkstra/A*, Google OR-Tools, and simulated annealing; (iv) implement QAOA in Qiskit and execute small instances on both noisy simulators and IBM Quantum hardware; and (v) rigorously compare solution quality against classical methods while analysing circuit depth, gate counts, and hardware fidelity/error behaviour. *The aim is deliberately twofold — scientific and educational:* to characterise honestly where quantum heuristics do and do not help on this class of problem at current hardware scales, while giving the investigator structured, hands-on mastery of the modern quantum-optimization toolchain.

1 Introduction & Motivation

1.1 The Dhaka congestion problem

Dhaka is among the most densely populated and traffic-congested cities on Earth. During peak hours, average vehicle speeds fall to little more than walking pace, and roads are frequently blocked or re-routed with little warning. For emergency services — ambulances, fire, and disaster response — every additional minute in transit has direct clinical and safety consequences. Time-critical conditions such as cardiac arrest, major trauma, and stroke are governed by a narrow therapeutic window (the so-called “golden hour”), so even modest reductions in response time can meaningfully change outcomes.

1.2 Why routing here is genuinely hard

Finding the shortest path between two points on a road graph is computationally easy — it is solved in polynomial time by Dijkstra's algorithm. The difficulty appears the moment the problem becomes realistic: dispatching multiple vehicles to multiple simultaneous incidents, respecting coverage and capacity, and reacting to time-varying congestion. This is the Vehicle Routing Problem (VRP) family, which is NP-hard: the number of candidate assignments grows combinatorially, and exact optimisation quickly becomes intractable. Combinatorial optimisation of exactly this kind is the natural target for quantum optimisation heuristics.

1.3 Why quantum, and why now

QAOA and QUBO map combinatorial problems directly onto the energy landscape of a quantum system, and Noisy Intermediate-Scale Quantum (NISQ) hardware is now openly accessible through cloud platforms such as IBM Quantum. **This proposal makes no claim of quantum advantage today.** Rather, it is a timely, well-scoped feasibility study: how close can QAOA get to the true optimum on small emergency-routing instances, what circuit resources and error rates does it incur, and how does it compare against mature classical methods? These are precisely the questions whose answers build durable skills and yield defensible, honest findings.

1.4 Learning framing

The project is intentionally sequenced so that each phase teaches a concrete layer of the stack — geospatial data and graphs, then classical optimisation, then QUBO modelling, then QAOA, then transpilation and noise, and finally real-hardware execution. A reader with some Qiskit familiarity can therefore treat this document simultaneously as a research plan and as a structured curriculum.

2 Problem Statement

Let the road network be a weighted directed graph $G = (V, E)$, where vertices V are intersections and edges E are road segments carrying a time-varying travel time t_e that reflects congestion. Two nested problems are studied:

- **(a) Single-vehicle fastest route.** Given a congestion-weighted graph, find the minimum-time path from a station s to an incident t . This is classically easy and is used as a learning warm-up and as a correctness check for the QUBO encodings.
- **(b) Multi-vehicle assignment & routing (the core).** Given k emergency vehicles at stations and m simultaneous incidents, assign vehicles to incidents and route them so as to minimise a weighted objective (e.g. total or maximum response time), subject to every incident being served and to coverage/capacity constraints. This is an NP-hard VRP/assignment problem and is the primary target for QUBO/QAOA.

Scope. Because NISQ hardware limits both qubit count and circuit depth, the quantum experiments deliberately target small instances (a few nodes / a handful of binary decision variables) that are solvable to optimality classically, so that QAOA solution quality can be measured exactly. The classical pipeline additionally handles realistic, city-scale instances, so the end-to-end system is demonstrated and the quantum results are properly contextualised.

3 Research Objectives & Questions

3.1 Objectives

- **01** Build a reproducible, congestion-aware Dhaka road-network dataset and a parameterised instance generator.

- **02** Implement and validate classical baselines: Dijkstra/A*, Google OR-Tools, and simulated annealing.
- **03** Derive and validate QUBO formulations for (a) shortest path and (b) small VRP/assignment, with tunable penalty weights.
- **04** Implement QAOA at variable depth p and run it on a noiseless simulator, a noisy (device-model) simulator, and IBM Quantum hardware.
- **05** Apply noise-aware transpilation and error mitigation, and quantify their effect.
- **06** Benchmark solution quality against classical methods and analyse resource scaling (qubits, depth, two-qubit gate count) and hardware error behaviour.

3.2 Research questions

- **RQ1** For small emergency-routing instances, how close does QAOA get to the known optimum as a function of depth p and choice of classical optimiser, and how does feasibility depend on penalty weights?
- **RQ2** How do transpiled circuit depth and two-qubit gate count grow with problem size under realistic mapping to IBM hardware coupling maps, and where does noise begin to dominate?
- **RQ3** Which error-mitigation techniques (measurement mitigation, dynamical decoupling, zero-noise extrapolation) deliver the best quality-per-cost on these circuits?
- **RQ4** At current hardware scales, what is the honest verdict on QAOA versus classical heuristics for this problem — by solution quality and by resource/wall-clock cost?

4 Background & Related Work

4.1 QUBO and the Ising model

A QUBO problem seeks a binary vector that minimises a quadratic form:

$$\text{minimise } f(\mathbf{x}) = \mathbf{x}^T \mathbf{Q} \mathbf{x}, \quad \mathbf{x} \in \{0,1\}^n$$

where $\mathbf{Q}$ is a real symmetric (or upper-triangular) matrix encoding costs on the diagonal and interactions off-diagonal. Any QUBO maps to an Ising Hamiltonian by the substitution

$$x_i = (1 - z_i) / 2, \quad z_i \in \{-1, +1\}$$

turning binary variables into ± 1 spins, so the minimum of the QUBO corresponds to the ground state of a problem Hamiltonian. Lucas (2014) gives Ising formulations for a wide range of NP problems, and Glover, Kochenberger & Du (2019) provide a practical QUBO modelling tutorial.

4.2 The Quantum Approximate Optimization Algorithm

QAOA (Farhi, Goldstone & Gutmann, 2014) prepares a parameterised state by alternately applying a cost unitary generated by the problem Hamiltonian H_C and a mixing unitary generated by a mixer H_M , for p layers:

$$|\psi(\gamma, \beta)\rangle = \prod_{i=1}^p e^{-i\beta_i H_M} e^{-i\gamma_i H_C} |+\rangle^{\otimes n}$$

A classical optimiser tunes the $2p$ angles (γ, β) to minimise the expected cost $\langle \psi | H_C | \psi \rangle$. Solution quality is commonly reported as an *approximation ratio*:

$$r = \langle \psi | H_C | \psi \rangle / E_{\text{opt}}$$

As p increases, QAOA can in principle approach the optimum, but deeper circuits accumulate more noise on NISQ hardware — the central tension this project measures.

4.3 Quantum routing and classical optimisation

Prior work has encoded TSP and VRP as QUBOs for quantum annealers (e.g. Feld et al., 2019) and explored QAOA on small combinatorial instances such as MaxCut. A recurring, honest finding across this literature is that today's hardware handles only very small instances and that error mitigation is essential. On the classical side, Dijkstra/A* solve shortest paths exactly, Google OR-Tools provides an industrial near-optimal routing solver, and metaheuristics such as simulated annealing (Kirkpatrick et al., 1983) optimise the QUBO directly — which makes SA the fairest “bridge” baseline, since it and QAOA minimise the same objective.

5 Methodology

5.1 Data acquisition & instance generation

- **Source.** Extract the drivable road graph for a Dhaka thana/district (for example Dhanmondi or Tejgaon) from OpenStreetMap using OSMnx. Nodes are intersections; edges carry segment length and an inferred speed, giving a travel time per edge.
- **Congestion.** Apply time-of-day speed multipliers (peak vs. off-peak) to edges to emulate congestion. Where live traffic feeds are unavailable, use transparent, documented synthetic congestion profiles.
- **Emergency structure.** Mark station nodes and generate incident sets; build vehicle-to-incident travel-time matrices via repeated Dijkstra for the assignment layer.
- **Reproducibility.** Fix random seeds, save the graph (GraphML) and matrices (CSV/NPZ), and expose an instance generator that produces problems of controllable size (n nodes, k vehicles, m incidents).

5.2 Mathematical formulation

Three formulations of increasing difficulty are used.

5.2.1 Shortest-path QUBO (warm-up)

With a binary variable x_e per edge, minimise total travel time subject to flow conservation encoded as a penalty (net flow $+1$ at the source, -1 at the sink, 0 elsewhere):

$$H = \sum_{e \in E} t_e x_e + P \sum_{v \in V} \left(\sum_{e \in \text{Out}(v)} x_e - \sum_{e \in \text{In}(v)} x_e - b_v \right)^2$$

5.2.2 Route-ordering (TSP-style) QUBO

For a single vehicle visiting a small set of waypoints, use the Lucas-style position encoding: $x_{i,p} = 1$ if node i is visited at tour position p . Constraint terms enforce that each node appears once and each position holds one node; the cost term sums edge weights between consecutive positions:

$$H_A = A \sum_i (1 - \sum_p x_{i,p})^2 + A \sum_p (1 - \sum_i x_{i,p})^2$$

$$H_B = \sum_{(i,j)} w_{ij} \sum_p x_{i,p} x_{j,p+1}, \quad H = H_A + H_B$$

with $A > B \cdot \max(w)$ so that feasibility dominates the objective. Note the qubit cost scales as n^2 in the number of nodes — the key reason only very small n fits on current hardware.

5.2.3 Assignment / VRP QUBO (core)

With $y_{v,c} = 1$ if vehicle v serves incident c , minimise total response time plus penalties enforcing that each incident is served exactly once and vehicle coverage/capacity is respected:

$$H = \sum_{v,c} w_{v,c} y_{v,c} + P_1 \sum_c (1 - \sum_v y_{v,c})^2 + P_2 (\text{capacity terms})$$

Conversion to a solver-ready form uses qiskit-optimization's QuadraticProgram and QuadraticProgramToQubo (automatic penalty handling), then to an Ising SparsePauliOp for QAOA. Penalty weights are tuned carefully: too small and the optimiser returns infeasible solutions; too large and the energy landscape develops small gaps that are hard to resolve under noise.

5.3 Classical baselines

- **Dijkstra / A* (NetworkX).** Exact shortest paths; ground truth for problem (a).
- **Google OR-Tools.** Industrial routing/assignment solver providing a strong near-optimal baseline for the VRP layer.
- **Simulated annealing.** Runs on the same QUBO (via dimod/neal or a custom sampler) — the fairest head-to-head comparison for QAOA.
- **Brute force.** Exhaustive search on the tiniest instances to obtain the exact optimum for approximation-ratio measurement.

5.4 Quantum approach (QAOA)

- **Ansatz.** Build H_C from the QUBO and construct the QAOA circuit (via qiskit-algorithms QAOA or a manual QAOAAnsatz) with p layers.
- **Optimiser.** Compare COBYLA and SPSA (SPSA is more robust under shot noise); use informed initialisation (e.g. linear-ramp / TQA schedules) to reduce the chance of stalling in poor local minima.
- **Execution.** Sample with SamplerV2 from qiskit-ibm-runtime; use Session/batch mode on hardware to reduce queue overhead.
- **Decoding.** Read out the most probable feasible bitstring, evaluate its cost, and repeat for statistics across seeds.

5.5 Noise-aware transpilation & error mitigation

- **Transpilation.** Compile with `optimization_level = 3` to the target backend's coupling map and native basis gates (CX/ECR) using SABRE routing; report depth and two-qubit gate count before and after.
- **Noise-first workflow.** Build an Aer noise model from the backend's calibration (T1/T2, gate and readout errors) and validate on the simulator before spending any hardware time.
- **Mitigation.** Use qiskit-ibm-runtime resilience options — readout/measurement mitigation, dynamical decoupling, Pauli twirling, and zero-noise extrapolation (ZNE) for expectation values — and compare their quality-versus-cost trade-offs.

5.6 Hardware execution plan (IBM Quantum)

- **Access.** Use the IBM Quantum Platform Open (free) plan for small runs. Monthly QPU minutes are limited, so validate exhaustively on the simulator first and reserve hardware for a few final, well-prepared instances.
- **Backend & sizing.** Choose a low-error backend and pick instances (realistically $\leq \sim 10$ –12 qubits) that fit within coherence and depth budgets.
- **Provenance.** Record calibration data at run time to support the error analysis.

6 Evaluation Plan & Metrics

Identical instances are run across all methods; results are reported as mean $\pm$ standard deviation over multiple seeds, presenting noiseless, noisy-simulator, and hardware results side by side.

Category	Metric	What it tells us
Solution quality	Approximation ratio, optimality gap %, feasibility rate, best-found vs. optimal	How good, and how often valid, the returned solutions are
Algorithmic	Convergence iterations; sensitivity to p , penalty weights, optimiser and initialisation	How stable and tunable QAOA is on this problem
Resource / scaling	Qubit count vs. problem size; transpiled depth; two-qubit gate count; #parameters	How circuit cost grows and where hardware limits bite
Hardware	Success probability; distribution fidelity (Hellinger / TV distance) vs. simulator; effect of each mitigation; QPU time	How much noise degrades results and what mitigation recovers
Classical comparison	Quality and wall-clock for Dijkstra / OR-Tools / SA on the same instances	The honest quantum-vs-classical verdict

7 Hands-On Learning Roadmap & Timeline

The plan spans roughly 12–16 weeks and is adjustable. Each phase pairs a concrete build task with a specific learning target and a deliverable, so progress is visible and skills compound.

Phase (weeks)	What you'll build	What you'll learn	Deliverable
0 — Setup (wk 1)	Install the toolchain; run a “hello quantum” circuit on simulator and hardware; create an IBM Quantum account	Primitives (SamplerV2), basic transpilation, the run workflow	Working environment + first hardware job
1 — Data & classical (wk 2–3)	Dhaka graph, instance generator, Dijkstra/A*, OR-Tools VRP, simulated annealing	Graph modelling, classical optimisation, OR-Tools	Classical solver + reproducible dataset
2 — QUBO (wk 4–5)	Shortest-path QUBO → TSP/assignment QUBO; validate vs. brute force; tune penalties	QUBO/Ising modelling, qiskit-optimization	Validated QUBO builder
3 — QAOA on simulator (wk 6–8)	QAOA implementation; sweep p, optimisers, initialisation; noiseless quality	Variational algorithms, cost/mixer Hamiltonians, classical optimisers	QAOA solver + baseline results
4 — Noise & transpilation (wk 9–10)	Device noise models; transpilation metrics; error mitigation on simulator	NISQ noise, the transpiler, resilience options	Noise & resource study
5 — Hardware (wk 11–12)	Run best small instances on IBM hardware; apply mitigation; gather fidelity data	Real-hardware workflow, calibration, QPU-time budgeting	Hardware results
6 — Analysis & write-up (wk 13–14+)	Benchmark quantum vs. classical; scaling & error analysis; final report	Scientific communication, reproducibility	Final report + clean code repo

8 Expected Outcomes & Contributions

- A reproducible, openly documented dataset and instance generator for Dhaka emergency routing.
- A clean, well-documented QUBO → QAOA pipeline with classical baselines — usable both as a learning artifact and as a benchmark harness.
- An honest empirical characterisation of QAOA solution quality and resource cost versus classical methods at NISQ scales, including the measured effect of error mitigation.
- For the investigator: end-to-end, hands-on competence across the full quantum-optimization stack.

Hypothesised finding (stated up front, honestly). At today's hardware scales, classical methods are likely to match or beat QAOA on both quality and speed. The contribution is therefore the rigorous, locally grounded (Bangladesh) case study, the reusable toolchain, and a resource/scaling analysis that points to exactly where hardware and algorithms must improve for this application.

9 Risks, Limitations & Mitigations

Risk / limitation	Mitigation
Hardware noise dominates results on small instances	Simulator-first workflow; keep instances tiny; apply error mitigation; report noiseless and noisy side by side
Limited free QPU minutes on the Open plan	Validate exhaustively on Aer; minimise and batch hardware submissions; reserve QPU for final runs
Qubit scaling (n^2 for TSP encoding) limits realism	Prefer assignment/decomposition formulations; keep n small; frame the quantum part explicitly as a feasibility study
Penalty-weight tuning is unstable	Systematic sweeps; use qiskit-optimization auto-penalty; always verify feasibility of returned solutions
Barren plateaus / optimiser stalls	Use SPSA, modest depth p , and informed initialisation (linear-ramp / TQA)
No live congestion feed for Dhaka	Use transparent synthetic congestion profiles and document all assumptions
Fast-moving Qiskit / IBM APIs	Pin package versions; follow official migration guides; re-verify quotas and APIs against current docs

10 Tools, Software & Resources

Layer	Tool	Purpose
Language	Python 3	Glue for the whole pipeline
Quantum core	Qiskit (core), qiskit-aer	Circuits, transpiler, high-performance + noisy simulation
Quantum runtime	qiskit-ibm-runtime	Hardware access; SamplerV2/EstimatorV2 primitives; resilience/mitigation
Modelling	qiskit-optimization, qiskit-algorithms	QUBO/QuadraticProgram modelling; QAOA and optimisers
Classical	Google OR-Tools; dimod/neal	VRP baseline; simulated annealing on the QUBO
Geospatial	OSMnx, NetworkX	Road-network extraction and graph algorithms

Layer	Tool	Purpose
Analysis	NumPy, pandas, Matplotlib	Experiments, statistics, and plots
Hardware	IBM Quantum Platform (Open plan)	Real superconducting QPUs via the cloud

10.1 Recommended learning resources

- IBM Quantum Learning — learning.quantum.ibm.com
- Qiskit documentation & migration guides — docs.quantum.ibm.com
- qiskit-optimization docs — qiskit-community.github.io/qiskit-optimization
- Google OR-Tools routing guide — developers.google.com/optimization/routing
- OSMnx docs — osmnx.readthedocs.io

Note on versions and quotas. Qiskit's primitive APIs and the IBM Quantum free-tier QPU allocation change frequently. Confirm the current Qiskit version, the SamplerV2/EstimatorV2 usage, and the Open-plan monthly minutes against the official documentation before finalising code.

11 References (selected)

-
- [1] Farhi, E., Goldstone, J., & Gutmann, S. (2014). A Quantum Approximate Optimization Algorithm. arXiv:1411.4028.
 - [2] Lucas, A. (2014). Ising formulations of many NP problems. *Frontiers in Physics*, 2:5.
 - [3] Glover, F., Kochenberger, G., & Du, Y. (2019). A Tutorial on Formulating and Using QUBO Models. arXiv:1811.11538.
 - [4] Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by Simulated Annealing. *Science*, 220(4598).
 - [5] Feld, S., et al. (2019). A Hybrid Solution Method for the Capacitated Vehicle Routing Problem Using a Quantum Annealer. *Frontiers in ICT*, 6:13.
 - [6] Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65.
 - [7] Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information*. Cambridge University Press.
 - [8] Perron, L., & Furnon, V. Google OR-Tools (software documentation).
 - [9] IBM Quantum documentation and IBM Quantum Learning (docs.quantum.ibm.com; learning.quantum.ibm.com).

Appendix A • Illustrative QAOA skeleton (Qiskit)

A compact, illustrative sketch of the QAOA path from a QUBO to a sampled solution. It is intended to convey the shape of the code, not to be a drop-in script; confirm exact imports and primitive usage against the current Qiskit version.

```
from qiskit_optimization import QuadraticProgram
from qiskit_optimization.converters import QuadraticProgramToQubo
from qiskit_algorithms import QAOA
from qiskit_algorithms.optimizers import COBYLA
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2 as Sampler

# 1. Build the model (costs + constraints) -> QUBO -> Ising
qp = QuadraticProgram()
# ... add binary vars, objective, and constraints for the routing instance ...
qubo = QuadraticProgramToQubo().convert(qp)
hamiltonian, offset = qubo.to_ising()

# 2. Pick a backend (simulator first; hardware later)
service = QiskitRuntimeService()
backend = service.least_busy(operational=True, simulator=False)
sampler = Sampler(mode=backend)

# 3. Run QAOA (start small: p = 1, then increase)
qaoa = QAOA(sampler=sampler, optimizer=COBYLA(), reps=1)
result = qaoa.compute_minimum_eigenvalue(hamiltonian)

# 4. Decode the most-probable bitstring into a route/assignment
solution = qubo.interpret(result.best_measurement['bitstring'])
print('cost =', result.best_measurement['value'] + offset)
```